TP JAVA · 2ÈME ANNÉE LICENCE GLSI · ISTIC

Mini-projets POO : cadre commun

Vous êtes développeur, le client change d'avis, votre code évolue.

Wissem Zeddini · Document de référence commun aux dix sujets de mini-projet

1. Le principe en une page

Dans une entreprise, un logiciel n'est jamais écrit d'un seul coup : le client exprime un besoin, l'équipe livre une première version, puis le client change d'avis et demande autre chose. Votre mini-projet fonctionne exactement comme cela.

- Vous recevez le **cahier des charges** d'un sujet : un client simulé, un vocabulaire métier, des règles à respecter.
- Vous travaillez **seul(e)**, en dehors des séances de TP, sur un logiciel en **mode console**.
- Le projet avance en **quatre versions** : v0.1, v0.2, v0.3 et v1.0. À chaque version arrive un **mail du client** qui ajoute ou modifie des demandes : votre code doit évoluer sans tout réécrire.
- À chaque version, vous livrez quelque chose **qui fonctionne** et que vous pouvez montrer en deux minutes.
- Le projet se termine par une **livraison complète** : code, fichier .jar, rapport, et démonstration de 5 minutes.

> **Pourquoi cette méthode ?**
> La POO sert surtout à **faciliter l'évolution**. Si votre code est bien conçu, ajouter un nouveau type d'objet (une trottinette, un nouveau type de chambre) ne demande presque aucun changement ailleurs. C'est ce que le client va tester, version après version.

2. Les règles du jeu

Sujet	Règle
Travail	Individuel. Plusieurs étudiants peuvent avoir le même sujet : chacun écrit **sa propre** solution.
Langage	Java standard, en console, sans bibliothèque externe et sans interface graphique.
Outil	Au choix : BlueJ, IntelliJ IDEA ou VS Code.
Fichiers	Une classe par fichier ; le nom du fichier est celui de la classe ; **tout doit compiler**.
Dépôt	À chaque version, au lieu indiqué par l'enseignant, avant la date limite.
Suivi	Un tableau de bord projeté en séance montre l'avancement de chacun (statuts et badges). Il ne classe jamais les derniers.

3. Le parcours en quatre versions

Version	Le client demande…	Notions	Vous livrez	Date limite
v0.1 Cadrage	« Voici mon besoin : montrez-moi que vous l'avez compris. »	Analyse, vocabulaire métier, UML	UML de départ (3 classes au moins), squelettes qui compilent, une classe Demo qui affiche un objet	.....
v0.2 Objets fiables	« Mes données doivent rester cohérentes. »	Encapsulation, constructeurs, validation	Attributs privés, getters et setters, une règle de validation au moins, toString(), une Demo avec 3 objets et un refus	.....
v0.3 La famille	« J'ai de nouveaux types à gérer. »	Héritage, redéfinition, polymorphisme	Une classe mère et 2 classes filles au moins, super(...), @Override, une liste d'objets, une boucle unique, UML mis à jour	.....
v1.0 Livraison	« Livrez-moi le produit. »	Défi métier, menu, assemblage	Défi métier fonctionnel, menu console, jeux d'essai, .jar et README, rapport, démonstration	.....

Les dates limites sont renseignées par l'enseignant d'après le calendrier des séances. Un retard n'empêche pas de continuer : la version suivante repart de votre dernier dépôt, ou du point de départ qui vous est fourni.

4. Ce que « terminé » veut dire

Une version est terminée quand **toute** sa liste de contrôle est cochée. Relisez-la avant de déposer.

v0.1 · Cadrage

- Vous avez choisi ou reçu un sujet et listé 5 à 8 mots du vocabulaire métier.
- Un UML de départ montre au moins 3 classes avec leurs attributs principaux.
- Les squelettes de classes **compilent**.
- Une classe Demo contient un main qui affiche un objet de votre sujet.

v0.2 · Objets fiables

- Tous les attributs sont **private**.
- Au moins un constructeur à paramètres, et un constructeur surchargé (ou un autre cas de this(...)).
- Des getters et au moins un **setter qui valide** la valeur reçue.
- Une méthode toString() ; la Demo crée au moins 3 objets et montre **un refus** d'une valeur invalide.

v0.3 · La famille

- Une classe mère porte ce qui est commun ; au moins 2 classes filles ajoutent leurs particularités.
- Les constructeurs des filles utilisent super(...) ; au moins une méthode est redéfinie avec @Override.
- Une liste (ou un tableau) du type commun contient des objets de types différents.
- **Une seule boucle** les traite tous, sans instanceof. L'UML est mis à jour.

v1.0 · Livraison

- Le défi métier du sujet fonctionne.
- Un menu console permet d'utiliser le logiciel sans toucher au code.
- La classe Demo rejoue vos jeux d'essai : valeurs normales, valeurs limites, erreurs.

- Le .jar se lance, le README explique comment, le rapport est rendu, la démonstration est prête.

Bonus : à partir de v0.3

- Une **classe abstraite** ou une **interface** (badge Abstrait), une **exception personnalisée** (badge Résilient), une **sauvegarde dans un fichier** (badge Archiviste).

5. Les demandes du client : les « user stories »

Chaque demande du client s'écrit sur le même modèle : **En tant que** [rôle], **je veux** [action], **afin de** [bénéfice]. Dans chaque sujet, chaque demande est associée à une notion du cours.

Demande du client	Notion mobilisée
« En tant que responsable des données, je veux que le nombre d'habitants ne soit jamais négatif, afin de garder des chiffres cohérents. »	Encapsulation : un setter qui valide
« En tant que gestionnaire, je veux afficher toutes les villes avec une seule boucle, qu'il s'agisse d'une ville ou d'une capitale, afin de ne pas écrire un code différent par type. »	Polymorphisme : une liste du type commun
« En tant qu'utilisateur, je veux comprendre ce que je dois saisir, afin de ne pas me tromper. »	Qualité : message avec unité et exemple

Chaque demande devient une ou plusieurs méthodes, **testées** par votre classe Demo.

6. Les piliers de la POO dans votre projet

Pilier	Ce qu'on attend	Comment le montrer
Encapsulation	Des attributs privés ; l'état n'évolue que par des méthodes qui protègent les règles du métier.	Une tentative de valeur invalide, refusée.
Héritage	Une classe mère qui porte ce qui est commun ; des classes filles qui ajoutent leurs particularités.	La duplication de code qu'on a évitée.
Polymorphisme	Une liste du type commun, une boucle unique, chaque objet exécutant sa version.	La ligne de code où le type réel de l'objet décide.
Abstraction (bonus)	Une classe abstraite ou une interface qui définit le contrat sans l'implémenter.	Pourquoi la classe de base ne peut pas être instanciée.

*L'abstraction est proposée en **bonus** : elle est valorisée par un badge, mais elle n'est pas exigée pour valider le projet.*

7. La qualité du code, comme dans le monde professionnel

- **Des noms explicites** : une classe porte un nom du métier, une méthode un verbe. Pas de `x`, `n1` ou `f()`.
- **Des messages clairs** : dire quoi saisir, avec l'unité et un exemple, par exemple « Entrez la durée en minutes (ex : 30) ».
- **Une méthode, une responsabilité.** Si vous recopiez du code, c'est souvent qu'une classe mère manque.
- **Des commentaires qui expliquent le pourquoi**, pas ce que la ligne fait déjà de façon évidente.

- **Les conventions Java** : `getX()` et `setX(...)` pour les accesseurs, `isX()` pour un booléen, classes en majuscule, méthodes en minuscule.
- **Une classe Demo** avec un main qui rejoue vos jeux d'essai.
- **L'argent** : évitez les nombres à virgule (float, double), qui font des erreurs d'arrondi ; préférez des entiers (par exemple des millimes).

8. Les livrables de la version finale

Livrable	Contenu	Format
Code source	Tous les fichiers .java, qui compilent.	Dossier compressé (.zip)
README	Comment compiler et lancer (3 lignes), liste des classes.	Fichier texte
Exécutable	Un fichier lancé avec java -jar.	.jar
UML	Le diagramme de classes final.	Image ou PDF
Rapport	2 à 3 pages (voir la section suivante).	PDF
Démonstration	5 minutes, 2 questions, une modification en direct.	En séance

9. Le rapport réflexif (2 à 3 pages)

Le rapport montre que vous avez **compris** votre conception, pas seulement qu'elle fonctionne.

1. Justification de l'architecture

Votre diagramme UML, et la raison pour laquelle vous avez choisi cette classe mère. Si vous avez traité l'abstraction : pourquoi une classe abstraite plutôt qu'une interface (ou l'inverse) ?

2. Analyse des piliers

- **Encapsulation :** un exemple concret de règle du métier protégée par un setter ou une méthode.
- **Héritage :** quelle duplication de code la classe mère a évitée.
- **Polymorphisme :** la **ligne de code exacte** où le type réel de l'objet détermine le comportement à l'exécution.
- **Abstraction (si traitée) :** pourquoi la classe de base ne peut pas être instanciée.

3. Journal de débogage et d'IA

Une erreur rencontrée (NullPointerException, ClassCastException, erreur de compilation…) : le message exact, votre hypothèse, le test, la correction. Et ce que l'IA vous a proposé, ce que vous avez vérifié, ce que vous avez refusé.

10. Utiliser l'IA intelligemment

- L'IA est **autorisée comme assistante** ; vous restez l'auteur du projet.
- Vous devez pouvoir **expliquer chaque ligne** que vous rendez.
- Toute aide de l'IA est **déclarée** dans votre journal.
- **Testez toujours** ce qu'elle propose : une IA peut produire du code qui plante. Exemple vu en TP : un code généré pour lire des dates échouait à cause d'un caractère réservé dans le motif.

> **Les trois questions du jury**
> **Pourquoi cette ligne ?** · **Que se passe-t-il si… ?** · **Comment l'avez-vous testé ?**
> Si vous savez répondre à ces trois questions pour chaque partie de votre code, l'IA ne vous a pas remplacé : elle vous a aidé.

11. La démonstration finale (5 minutes)

Durée	Étape
1 min	Le contexte : qui est le client, quel est son besoin.
2 min	Le logiciel en marche, depuis le menu console.
1 min	Le moment polymorphisme : montrer la ligne de code où le type réel décide.
1 min	Deux questions, puis **une modification en direct** (par exemple ajouter un nouveau type d'objet).

Aucun diaporama n'est nécessaire : la console suffit. Un **Demo Day** collectif clôture la dernière séance.

12. Le barème proposé (sur 20, modifiable)

Critère	Points
Conception et UML	3
Encapsulation et qualité du code	4
Héritage et polymorphisme	4
Versions livrées dans les délais	3
Rapport et journal de débogage	3
Démonstration et soutenance	3
Total	**20**
Bonus (abstraction, exceptions, fichier)	jusqu'à +2

13. Les badges

Les badges **valorisent** un travail remarquable ; ils ne remplacent pas la note.

Badge	On le gagne pour…
Architecte	Un UML clair, cohérent avec le code, mis à jour à chaque version.
Clean Coder	Un code lisible, sans duplication, aux noms explicites.
Bug Hunter	Un journal de débogage exemplaire.
Polymorphe	Une boucle polymorphe élégante, sans instanceof.
Démo Star	Une démonstration claire, sans accroc, avec une modification en direct réussie.
Abstrait	Une classe abstraite ou une interface bien justifiée.
Résilient	Une exception personnalisée bien utilisée.

Badge	On le gagne pour…
Archiviste	Une sauvegarde et un rechargement par fichier.

14. Fabriquer et lancer votre fichier .jar

Un fichier **.jar** regroupe votre programme compilé : une seule chose à livrer, lancée par une seule commande.

Avec BlueJ

- Compilez : toutes les classes doivent être sans erreur.
- Menu **Project**, puis **Create Jar File…** (l'intitulé peut varier selon la version).
- Choisissez la classe qui contient le `main` (celle du menu), donnez un nom au fichier, validez.

Avec le terminal

Créez avec un éditeur de texte un fichier `manifest.txt` qui contient la ligne ci-dessous, **suivie d'un retour à la ligne** (remplacez Menu par la classe qui contient votre main) :

```
Main-Class: Menu
```

Puis, dans le dossier du projet :

```
javac *.java
jar cfm MonProjet.jar manifest.txt *.class
java -jar MonProjet.jar
```

> **Deux pièges à connaître**
>
> **La version de Java.** Un fichier compilé avec une version récente de Java peut être refusé par une version plus ancienne (erreur « UnsupportedClassVersionError »). Compilez avec la version qui lancera le fichier, ou installez un JDK plus récent.
>
> **Le double-clic.** Un programme console lancé par double-clic se ferme aussitôt : lancez-le depuis un terminal avec `java -jar`.

15. Pour aller plus loin : la persistance

Tant que votre programme tourne, vos objets vivent dans la **mémoire vive** : dès que vous le fermez, ils disparaissent. Si le client demande « je ferme le logiciel, je le rouvre, et mes données doivent être encore là », il faut les **écrire dans un fichier** (texte ou CSV) à la fermeture et les relire au démarrage. Ce n'est pas une question de nombre d'objets, mais de besoin de conservation. C'est le badge Archiviste.

16. Mini-glossaire

Terme	Sens
Cahier des charges	Le document qui décrit le besoin du client.
User story	Une demande du client écrite en une phrase : en tant que… je veux… afin de…
Version (release)	Un état du logiciel qui fonctionne et qu'on peut livrer.
Démo	La présentation du logiciel en marche, sans diaporama.
Liste de contrôle (Definition of Done)	Les conditions pour qu'une version soit déclarée terminée.
Revue de code	Relire le code de quelqu'un pour le comprendre et l'améliorer.

Annexe : ma feuille de route personnelle

Nom et prénom : ..... Sujet : .....

Version	Date limite	Livrée ?	Remarques
v0.1 · Cadrage			
v0.2 · Objets fiables			
v0.3 · La famille			
v1.0 · Livraison			
Bonus			